



**UNIVERSIDAD
DE GRANADA**

Informática Gráfica (curso 2026-27)

Guiones de Prácticas

Dpt. Lenguajes y Sistemas Informáticos.
ETSI Informática y de Telecomunicación.

Índice

1. Escena básica y modos de visualización	4
1.1. Objetivos	4
1.2. Requisitos previos	4
1.3. Actividades	4
1.3.1. Crear un nuevo proyecto Godot	5
1.3.2. Crear un cubo en la escena	6
1.3.3. Añadir ejes de coordenadas	7
1.3.4. Añadir cámara	8
1.3.5. Asignar materiales	9
1.3.6. Añadir luz	10
1.3.7. Crea una pirámide	11
1.3.8. Cambiar el material para que se vea rojo.	13
1.3.9. Controlar una cámara orbital por teclado y ratón	14
1.4. Entrega de la práctica	14

Lista de figuras

Figura 1	Ventana para crear un proyecto nuevo	6
Figura 2	Creación de la escena y el nodo raíz	6
Figura 3	Añadiendo cubo	7
Figura 4	Ejes de coordenadas	8
Figura 5	Cámara añadida a la escena	8
Figura 6	Asignación de material	9
Figura 7	Cambiando el color	9
Figura 8	Ejecución tras añadir cámara	10
Figura 9	Fuente de luz direccional añadida a la escena	10
Figura 10	Ejecución con cámara y luz direccional (aspecto más realista).	11
Figura 11	Código del <i>script</i> para la pirámide de la práctica 1 (en el archivo <code>piramide.gd</code>)	11
Figura 12	Creación de un <i>script</i> para la pirámide	13
Figura 13	Ejecución con pirámide sobre el cubo.	13
Figura 14	Ejecución con pirámide sobre el cubo, con material rojo.	14
Figura 15	Código del <i>script</i> para la cámara orbital (en el archivo <code>camara_3d_orbital_simple.gd</code>)	14

Práctica 1.

Escena básica y modos de visualización.

1.1. Objetivos

El objetivo de esta práctica es familiarizarse con el entorno de desarrollo de Godot, el sistema de nodos y la creación de escenas 3D simples. Los estudiantes aprenderán a:

- Crear una escena 3D con geometría básica (cubo y pirámide).
- Aplicar distintos materiales a los objetos.
- Implementar cambios de modo de visualización usando entrada por teclado.
- Controlar la cámara mediante teclas.

1.2. Requisitos previos

Para realizar esta práctica no se requiere experiencia previa en motores de juego ni gráficos 3D, pero antes de hacerla debes asegurarte de:

- Tener instalada como mínimo la versión 4.7.2 de Godot. Se puede descargar de la página oficial:

<https://godotengine.org/download>

hay versiones para Windows, Linux o macOS (cualquiera puede usarse para estas prácticas). La versión estable más reciente es la 4.7.2 (Agosto de 2026). No es necesaria la descarga de la versión **.net** del *engine*, basta con la versión estándar.

- Conocer los conceptos básicos de programación estructurada y orientada a objetos (estructuras de control y de datos básicas, funciones, clases).
- Descargar los archivos de script (extensión **.gd**) que se usarán en esta práctica, son los siguientes:
 - **ejes3D.gd**: script para dibujar ejes de coordenadas en la escena.
 - **piramide.gd**: script para crear una pirámide mediante código.
 - **camara_3d_orbital_simple.gd**: script para controlar una cámara orbital con teclado y ratón.

1.3. Actividades

En las subsecciones a continuación se detallan cada una de las actividades a realizar, son las siguientes:

1. Crear un nuevo proyecto Godot
2. Crear un cubo en la escena
3. Añadir ejes de coordenadas
4. Añadir cámara
5. Asignar materiales

6. Añadir luz
7. Crea una pirámide
8. Cambiar el material para que se vea rojo.
9. Controlar una cámara orbital

1.3.1. Crear un nuevo proyecto Godot

Se debe de dar estos pasos:

1. Abrir Godot y seleccionar la opción *Crear nuevo proyecto* en la ventana de inicio.
2. Se abre una ventana emergente para configurar algunas características del proyecto. En esta ventana hay que:
 - Asignar un nombre y una carpeta de destino (ver figura 1).
 - Seleccionar tipo de renderizador por defecto: seleccionamos *Forward+*

Al pulsar en el botón *Crear* se creará el proyecto.

3. Se abre la ventana del editor y en ella se crea el nodo raíz de la escena, de tipo 3D. Para ello se pulsa sobre el botón *Escena 3D* que hay debajo del rótulo *Crear Nodo Raíz*.
4. Se renombra el nodo raíz, y se le asigna el nombre *Escena Principal*. El editor quedará como se ve en la figura 2.
5. Definir la escena principal del proyecto, para ello se usa en el menú la opción *Proyecto ⇒ Configuración del proyecto*, en la ventana emergente se selecciona la pestaña *General* y en el apartado *Aplicación ⇒ Ejecutar* hay un campo que es *Escena principal*. Se pulsa sobre el botón de carpeta que hay a la derecha y se selecciona la escena que se acaba de crear (el archivo `node_3d.tscn`).
6. Guardar la escena para no perder el trabajo hecho hasta este punto, para eso usamos en el menú, la opción: *Escena ⇒ Guardar escena*. En la venta emergente dejamos el nombre propuesto (`node_3d.tscn`) y pulsamos *Guardar*.

En este punto se debe de haber creado una carpeta con el proyecto Godot, en dicha carpeta debe haber un archivo `project.godot` (es el archivo de configuración de proyecto, se puede abrir en adelante con Godot), además debe estar el archivo `node_3d.tscn`.

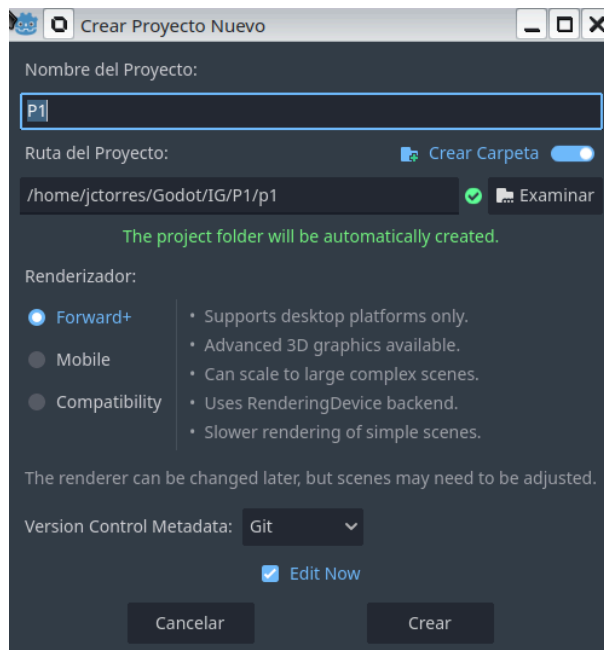


Figura 1 : Ventana para crear un proyecto nuevo

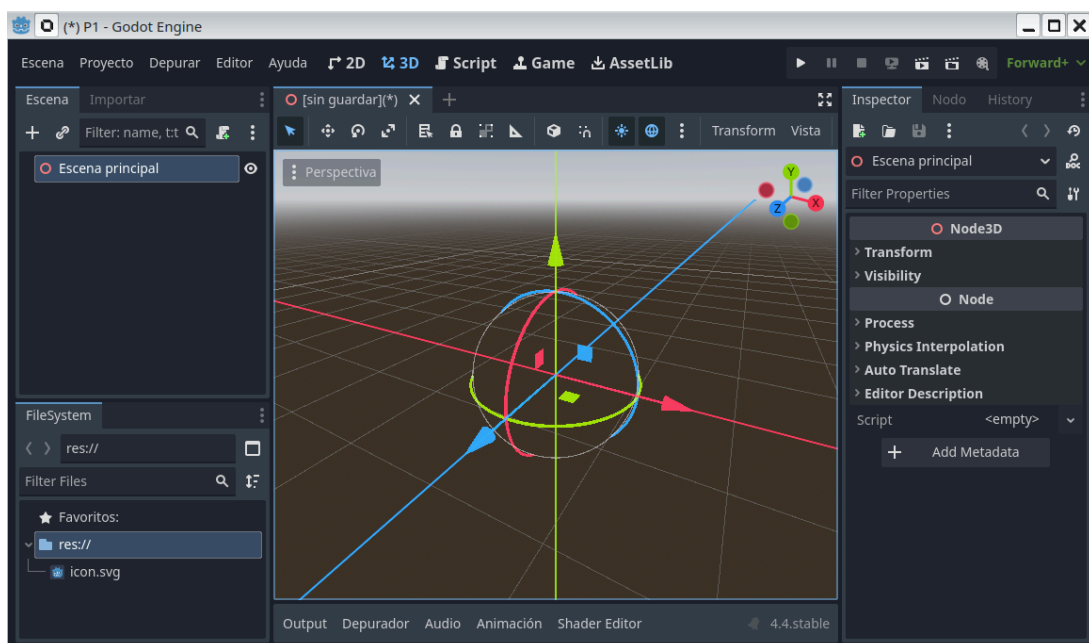


Figura 2 : Creación de la escena y el nodo raíz

1.3.2. Crear un cubo en la escena

Se añadirá un objeto de tipo malla con forma de cubo, para ello se usará la clase **CubeMesh** de Godot, que ya está predefinida en el motor.

1. Añadir un nodo de tipo **MeshInstance3D** como hijo del nodo raíz. Para ello se pincha con el botón derecho en el nodo raíz, se usa la opción *Añadir nodo hijo*, en la ventana emergente se busca el tipo **MeshInstance3D** y finalmente se pulsa *Crear*.

2. Asigna una malla **BoxMesh** en el campo **Mesh** del inspector. Para ello se selecciona el nodo, de forma que en el inspector (al lado derecho de la ventana) aparecerán sus propiedades, entre las cuales está el campo **Mesh**. A la derecha de **Mesh** se selecciona **BoxMesh** para darle forma de caja o cubo.
3. Renombrar el nodo como **Cubo**.
4. Moverlo a la posición (0, 0.5, 0). Para ello se debe de editar, en el inspector, el campo **Position**, dentro de **Transform**, en la sección **Node3D** (ver figura 3, a la derecha, abajo).

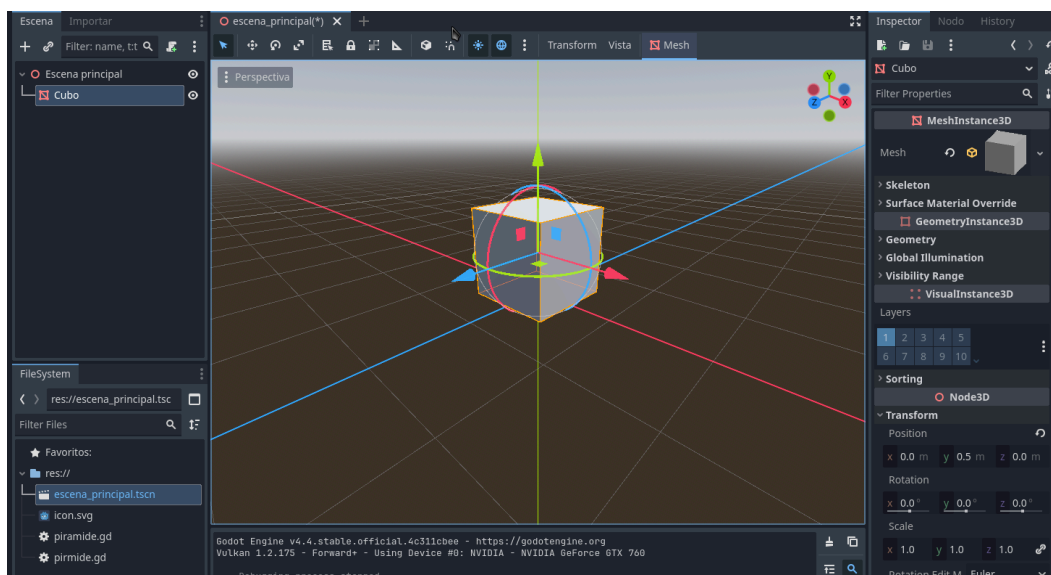


Figura 3 : Añadiendo cubo

1.3.3. Añadir ejes de coordenadas

Se añade un nodo 3D con un script que al inicio genera mallas en los ejes de coordenadas, parecidas a las que aparecen en la vista 3D del editor, sirve para entender visualmente mejor la posición y tamaño de los objetos de la escena cuando se ejecuta la aplicación. Para ello se dan los siguientes pasos:

1. Añadir un nodo de tipo **Node3D** (como hijo del nodo raíz) y renombrarlo como **Ejes3D**.
2. Copiar el archivo **ejes3D.gd** en la carpeta del proyecto Godot (la misma carpeta donde está el archivo **project.godot**). Comprueba que ese script aparece en el panel **Sistema de archivos** del editor.
3. Añadir a ese nodo un **script**, se hace pulsando sobre el nodo con el botón derecho y seleccionando **Añadir script**, después se abre una ventana emergente y en ella pinchamos en el símbolo de la carpeta y seleccionamos el archivo **ejes3D.gd**.

En la figura 4, se observan los ejes (tienen colores a los que no le afecta la iluminación)

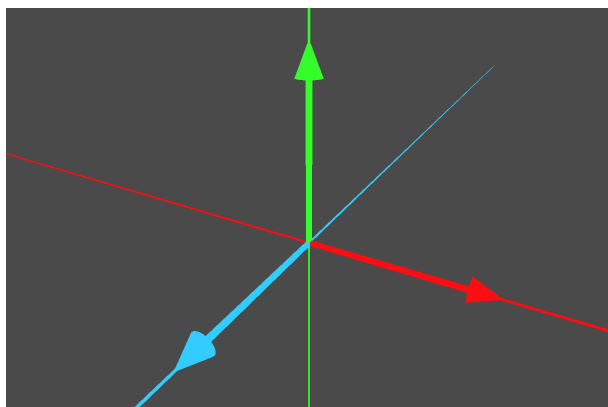


Figura 4 : Ejes de coordenadas

1.3.4. Añadir cámara

Para poder visualizar la escena, es necesario añadir una cámara que define la posición y orientación del observador virtual que se usa para producir la imagen. Esta cámara es muy simple, luego se añadirá otra más completa.

1. Añadir un nodo de tipo **Camera3D** como hijo del nodo raíz.
2. Colocararlo en una posición adecuada para ver toda la escena, por ejemplo en (1.5, 1.5, 2.0) y hacer que la cámara mire al origen (0, 0, 0). Esto se puede hacer añadiéndole al nodo de la cámara un script nuevo (vacío), y después en el archivo de script escribir este código:

```
extends Camera3D

func _ready():
    position = Vector3( 1.5, 1.5, 2.0 )
    look_at( Vector3( 0.0, 0.0, 0.0 ), Vector3.UP )
```

3. Ejecutar. El cubo aparece más o menos en el centro de la ventana, de color gris oscuro.

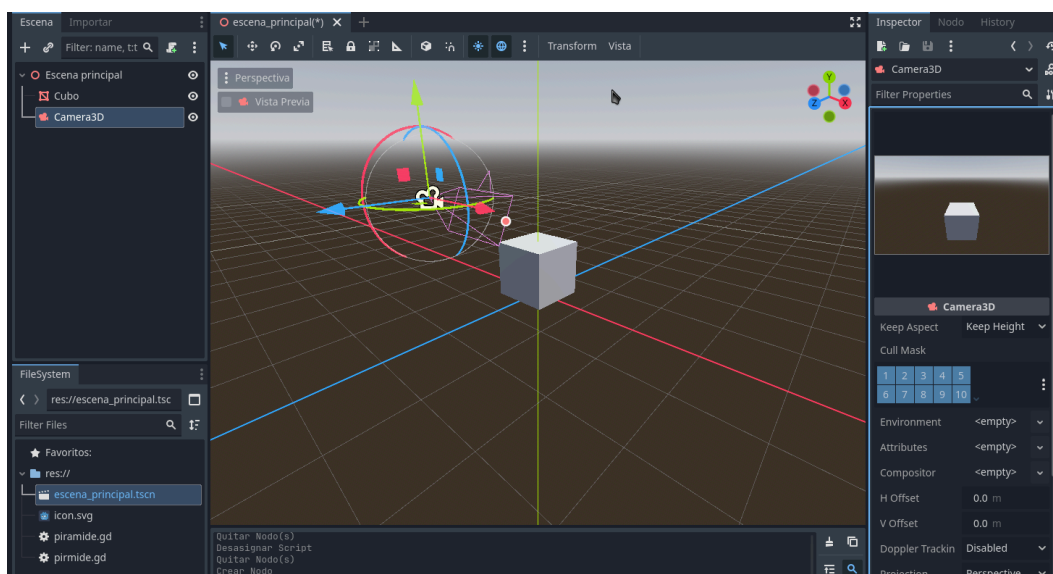


Figura 5 : Cámara añadida a la escena

1.3.5. Asignar materiales

1. Al seleccionar el cubo, en el inspector aparece una entrada llamada *Surface Material Override*, la cual permite seleccionar el tipo de material, lo que determina el aspecto visual del objeto. Aquí se debe seleccionar la opción **StandardMaterial3D** (ver figura 6).
2. Pulsar en la esfera y modificar el color del **Albedo**, asignar un color Amarillo (ver figura 7).
3. Ejecutar. Ahora el cubo aparece de color amarillo oscuro (ver figura 8).

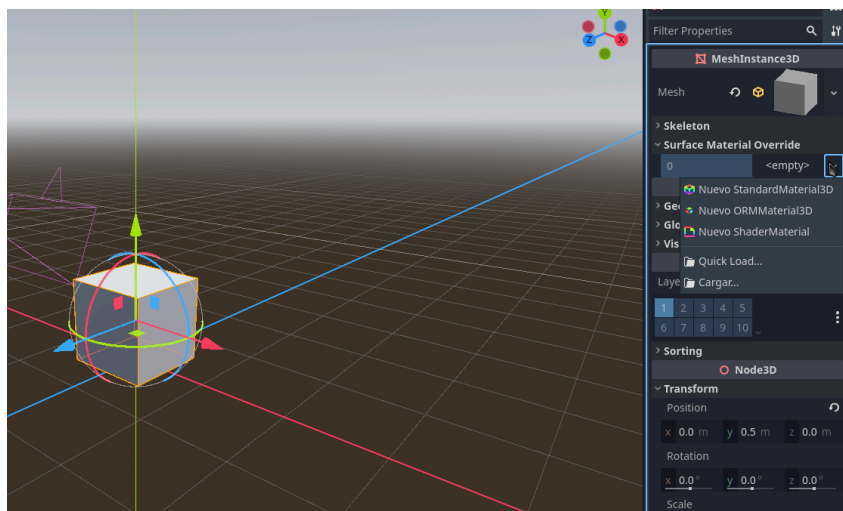


Figura 6 : Asignación de material

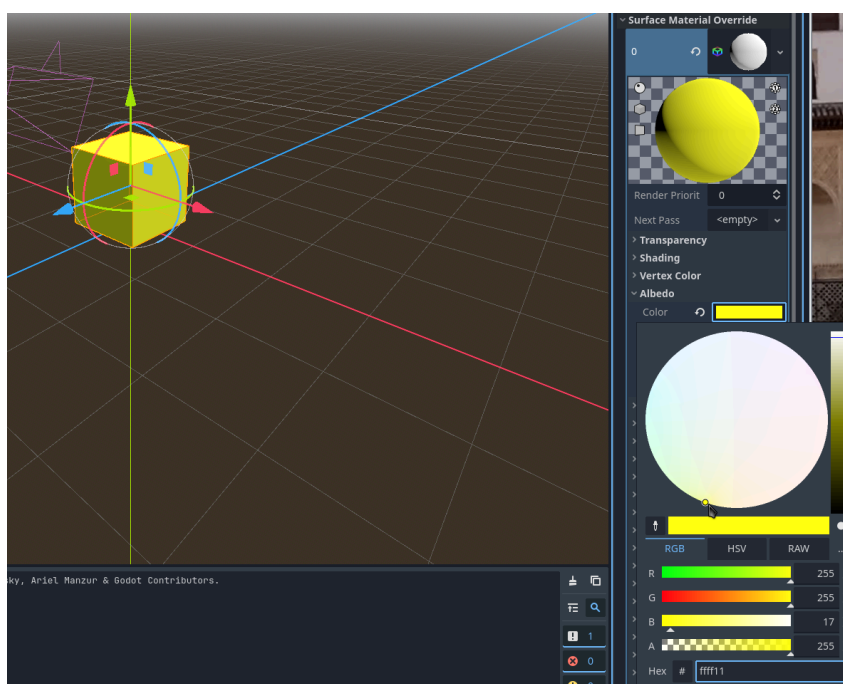


Figura 7 : Cambiando el color

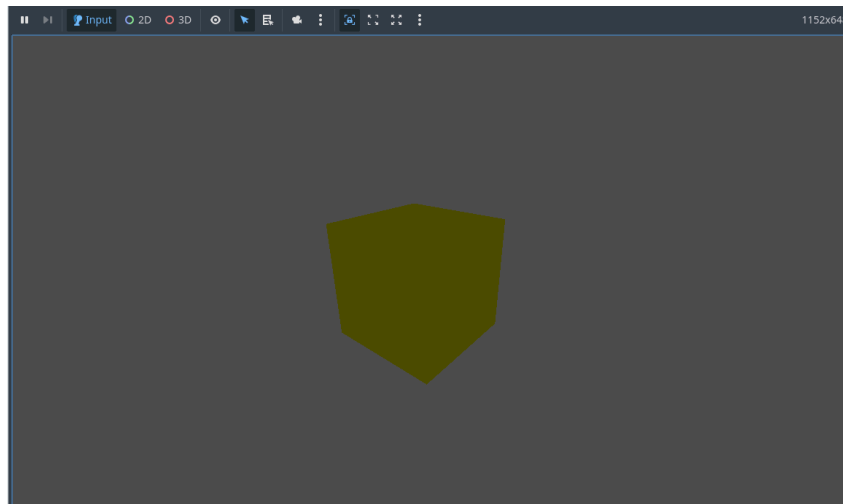


Figura 8 : Ejecución tras añadir cámara

1.3.6. Añadir luz

Para que los objetos tengan una apariencia realista, es necesario añadir al menos una fuente de luz a la escena (hasta entonces se ven muy oscuros). Se añadirá una luz direccional, que simula la luz del sol, y que ilumina todos los objetos de la escena de forma uniforme.

1. Añadir un nodo de tipo `DirectionalLight3D` como hijo del nodo raíz.
2. Colocar la fuente fuera del origen para distinguir su representación de la del cubo. Para ello podemos arrastrar la fuente de luz en la vista 3D, pulsando sobre los tres cuadrados de colores rojo, verde y azul, que están junto al centro de la esfera que la representa. Cada cuadrado permite mover la fuente en el plano donde reside (en dos direcciones).
3. Rotar la fuente para que ilumine el cubo de forma que las tres caras tengan distinta tonalidad, para ello se pulsa en la vista 3D sobre los tres arcos de colores rojo, verde y azul que rodean la posición de la fuente. Cada uno de ellos permite rotar sobre un eje (el X, el Y o el Z). Queda más o menos como en la figura 9.
4. Ejecutar. Cada cara del cubo se ve con una luminosidad distinta, según su orientación relativa a la fuente de luz (ver figura 10).

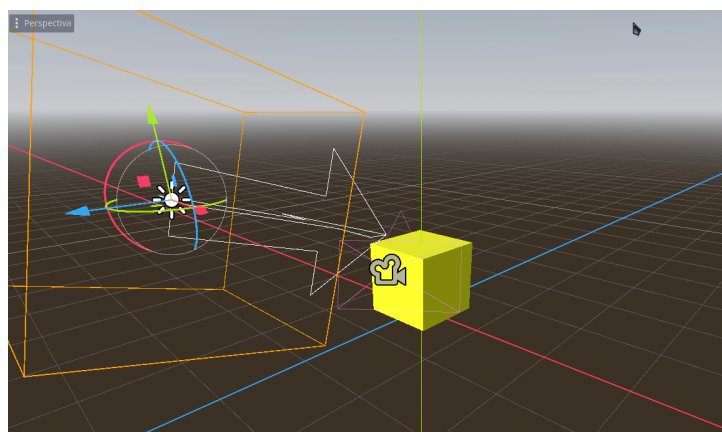


Figura 9 : Fuente de luz direccional añadida a la escena

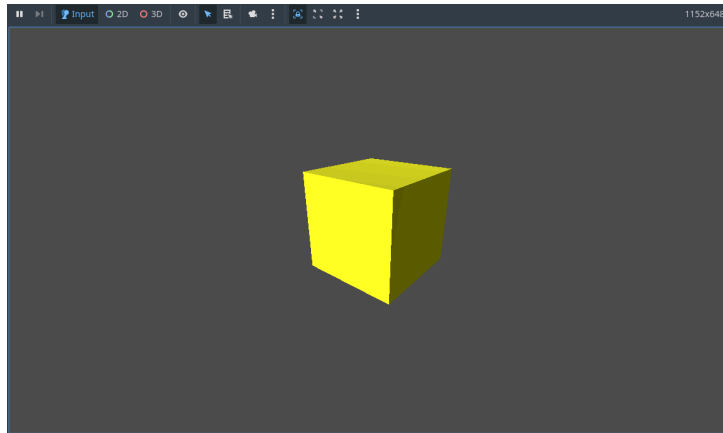


Figura 10 : Ejecución con cámara y luz direccional (aspecto más realista).

1.3.7. Crea una pirámide

Vamos a crear una pirámide de base cuadrada y la situamos sobre el cubo. El aspecto novedoso es que ahora la forma (la geometría) de este objeto se define mediante código (con un script), en lugar de simplemente usar una forma ya predefinida en Godot, como se hizo con el cubo.

Este script puede verse en la figura 11, contiene una función (`crear_piramide`) que crea la geometría de la pirámide. En el método `_ready()` (que se ejecuta una sola vez al inicio de la ejecución de la aplicación) se llama a esa función para crear la pirámide y añadirla a la escena. El código depende de un parámetro (de tipo `float`), llamado `altura`, que tiene un valor por defecto, pero que se puede modificar desde el inspector (en el editor), y que determina la altura de la pirámide.

Los pasos a dar son:

1. Crear un nodo de tipo `Node3D`, como hijo del nodo raíz y renómbra-lo como *Piramide*.
2. Asocia al script con el código, y que tienes disponible en el archivo `piramide.gd` (ver figura 12). Para ello, copia el archivo fuente gdscrip-t en la carpeta del proyecto (verás que al hacerlo, ese script te aparece en el panel *Sistema de archivos* del editor). Después asigna el script al nodo de la pirámide.
3. Posiciona la pirámide sobre el cubo, en las coordenadas (0, 1, 0). Para ello, modifica su posición en el inspector, en el campo `position` dentro de la sección *Transform*.
4. Modifica el parámetro `altura` en el Inspector asignándole valor 0.5.
5. Ejecuta. La pirámide aparece sobre el cubo (ver figura 13).

```

extends Node3D

@export var altura: float = 1.5

func _ready():
    var piramide = crear_piramide(altura)
    var mesh_instance = MeshInstance3D.new()
    mesh_instance.mesh = piramide
    add_child(mesh_instance)

func crear_piramide(h: float) -> ArrayMesh:
    var st = SurfaceTool.new()
    st.begin(Mesh.PRIMITIVE_TRIANGLES)

    # Coordenadas de la base (cuadrado centrado en el origen, lado 1)
    var p1 = Vector3(-0.5, 0, -0.5)
    var p2 = Vector3( 0.5, 0, -0.5)
    var p3 = Vector3( 0.5, 0,  0.5)
    var p4 = Vector3(-0.5, 0,  0.5)

    var apex = Vector3(0, h, 0) # cúspide

    # Caras laterales (triángulos)
    _add_triangulo(st, p1, p2, apex)
    _add_triangulo(st, p2, p3, apex)
    _add_triangulo(st, p3, p4, apex)
    _add_triangulo(st, p4, p1, apex)

    # Base (dos triángulos)
    _add_triangulo(st, p1, p3, p2, Vector3.DOWN)
    _add_triangulo(st, p1, p4, p3, Vector3.DOWN)

    return st.commit()

# Añade triángulo y calcula normal automáticamente
func _add_triangulo(st: SurfaceTool,
                    a: Vector3, b: Vector3, c: Vector3,
                    normal_override: Vector3 = Vector3.ZERO):
    var normal = normal_override
    if normal == Vector3.ZERO:
        normal = Plane(a, b, c).normal
    st.set_normal(normal)
    st.add_vertex(a)
    st.add_vertex(b)
    st.add_vertex(c)

```

Figura 11 : Código del *script* para la pirámide de la práctica 1 (en el archivo `piramide.gd`)

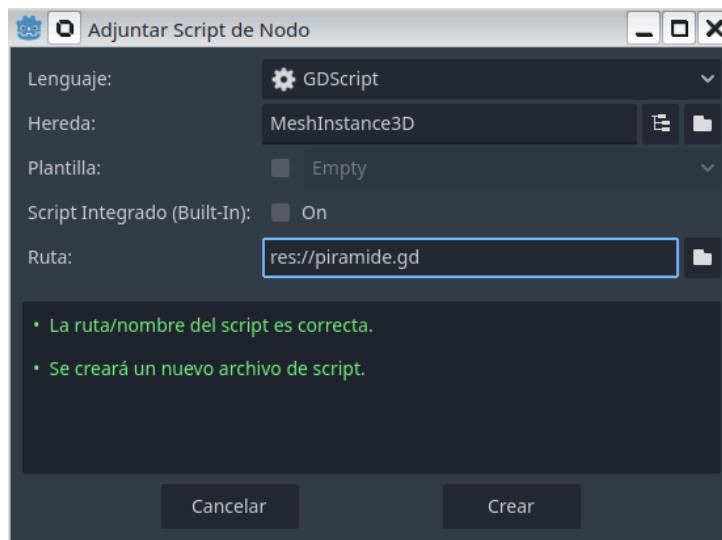


Figura 12 : Creación de un script para la pirámide

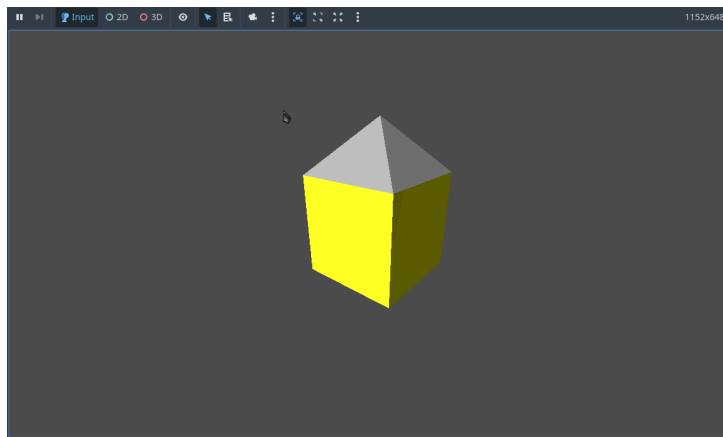


Figura 13 : Ejecución con pirámide sobre el cubo.

1.3.8. Cambiar el material para que se vea rojo.

Se le cambiará el material a la pirámide para que se vea de color rojo uniforme.

1. Edita la función `_ready()` del script anterior para añadir la creación del material (variable `material`), asignación de su color (rojo-naranja), y asignación del material a la propiedad `material_override` del nodo `mesh_instance`. Al final el código debe de quedar como se ve aquí:

```
func _ready():
    var piramide = crear_piramide(altura)
    var mesh_instance = MeshInstance3D.new()
    var material = StandardMaterial3D.new()
    mesh_instance.mesh = piramide
    material.albedo_color = Color(1.0, 0.1, 0.2) # Rojo-Naranja
    mesh_instance.material_override = material
    add_child(mesh_instance)
```

2. Ejecuta, ahora la pirámide se verá de color rojo (ver figura 14).

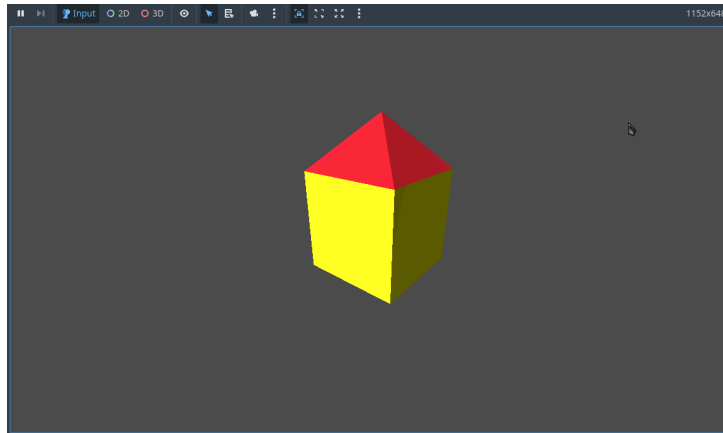


Figura 14 : Ejecución con pirámide sobre el cubo, con material rojo.

1.3.9. Controlar una cámara orbital por teclado y ratón

Vamos a hacer que la cámara pueda girar alrededor del origen para mostrar la escena desde diferentes posiciones, que también se puede acercar o alejar del origen, y que eso se pueda controlar con ratón o teclado. El código fuente de este script puede verse en la figura 15, y está en el archivo `camara_3d_orbital_simple.gd` que se proporciona con los materiales de la práctica.

Para orbitar el ratón hay dos opciones:

- Teclas de cursor
- Arrastrar el ratón con su botón derecho pulsado.

Para acercar o alejar la cámara al origen también hay varias opciones:

- Teclas `+` o `-`
- Teclas de página anterior o página siguiente.
- Mover la rueda del ratón en las dos direcciones.

Los pasos a dar son:

1. Borra el nodo `Camera3D` que se añadió antes, después borra también el script que se le asignó (`camera_3d.gd`), que ya no es necesario.
2. Añade un nuevo nodo de tipo `Camera3D` como hijo del nodo raíz, y renómbralo como `Camara3DOrbital`.
3. Copia el archivo `camara_3d_orbital_simple.gd` en la carpeta del proyecto Godot (la misma carpeta donde está el archivo `project.godot`), verifica que aparece en *Sistema de archivos* en el editor.
4. Añade al nuevo nodo de cámara el script que has creado.
5. Ejecuta y comprueba que puedes controlar la cámara con el teclado y el ratón.

1.4. Entrega de la práctica

- Subir la carpeta del proyecto Godot con todos los archivos necesarios comprimido en un archivo `.zip`.
- Analiza los scripts y contesta el test de la práctica en Prado.

```

extends Camera3D

const at := 2.5  ## ángulo de rot. con teclas (en grados x pulsación)
const ar := 0.5  ## ángulo de rot. con ratón (en grados x pixel)
var bdrp := false ## botón derecho del ratón presionado sí/no
var dz := 3.0  ## distancia en Z de la cámara al origen
var dxy := Vector2( 0.0, 0.0 ) ## ángulos hor. y vert. (en grados)

func _actualiza_transf_vista( ) -> void :
    var ahr := ((45.0+float(dxy.x))*2.0*PI)/360.0 ## ang.horiz.radi.
    var avr := ((30.0+float(dxy.y))*2.0*PI)/360.0 ## ang.vert.radi.
    var tras := Transform3D().translated( Vector3( 0.0, 0.0, dz ))
    var rotx := Transform3D().rotated( Vector3.RIGHT, -avr )
    var roty := Transform3D().rotated( Vector3.UP, ahr )
    transform = roty*rotx*tras  ## actualiza transform del nodo cámara

func _ready() -> void : ## se ejecuta una vez al inicio
    _actualiza_transf_vista() ## inicializa la vista al inicio

func _input( event : InputEvent ) : ## procesa evento de entrada
    var av : bool = true ## actualizar vista sí/no

    if event is InputEventKey and event.pressed: ## teclas
        match event.keycode:
            KEY_UP: dxy += Vector2( 0, -at )
            KEY_DOWN: dxy += Vector2( 0, +at )
            KEY_RIGHT: dxy += Vector2( -at, 0 )
            KEY_LEFT: dxy += Vector2( at, 0 )
            KEY_MINUS, KEY_PAGEDOWN, KEY_KP_SUBTRACT: dz *= 1.05
            KEY_PLUS, KEY_PAGEUP, KEY_KP_ADD: dz = max( dz/1.05, 0.1 )
            _: av = false
    elif event is InputEventMouseButton: ## botón ratón o rueda
        match event.button_index:
            MOUSE_BUTTON_RIGHT: bdrp = event.pressed ; av = false
            MOUSE_BUTTON_WHEEL_DOWN: dz *= 1.05
            MOUSE_BUTTON_WHEEL_UP: dz = max( dz/1.05, 0.1 )
            _: av = false
    elif event is InputEventMouseMotion and bdrp: ## movim. ratón
        dxy += ar * Vector2( -event.relative.x, event.relative.y )
    else: ## otros tipos de eventos (no hace nada)
        av = false ## (no actualizar transf)

    if av: _actualiza_transf_vista( )

```

Figura 15 : Código del *script* para la cámara orbital (en el archivo *camara_3d_orbital_simple.gd*)